

PipeANN 混合过滤框架正式集成实施计划

GitHub Copilot

2026 年 4 月 23 日

1 文档目标

本文档只保留可实施内容，用于指导 PipeANN 将现有的 hybrid 搜索能力从 `hybrid_rebuild_context` 迁移到正式源码树，并在正式接线与验证完成后删除整个 `hybrid_rebuild_context` 目录。本文档不再记录代码审查结论，不再区分“已有能力”和“缺失能力”，只定义后续工程实施的目标、模块落点、阈值机制和验证步骤。

2 实施范围

本次实施范围限定为以下四项：

1. 将 hybrid 搜索框架从实验目录迁入正式代码目录，即迁入 `include/`、`src/` 和 `tests/`；
2. 在正式查询接口中新增统一 hybrid 查询入口，使 `prefilter` 与 `graph-only` 两条路径由同一套运行时逻辑调度；
3. 将路径切换机制统一为候选规模单阈值 τ_m ，并在第一次索引构建完成后通过一次轻量 `benchmark` 生成初始阈值；
4. 将阈值重定位的唯一触发条件定义为向量数量变化超过 10%，并要求重定位在后台单线程执行，且每条 `benchmark` 查询开始前都要检测前台负载。

标签变化不作为阈值重定位触发条件。标签更新只负责维护查询时的候选规模计算输入，不负责触发重新 `benchmark`。

3 总体实施目标

本次集成完成后，PipeANN 中的过滤查询应具备如下统一执行流程：

1. 查询到达正式查询入口；
2. 运行时根据当前过滤条件计算候选规模 $m(F) = |S_F|$ ；
3. 将 $m(F)$ 与候选规模阈值 τ_m 进行比较；
4. 若 $m(F) \leq \tau_m$ ，则进入 `prefilter` 路径；
5. 若 $m(F) > \tau_m$ ，则进入 `graph-only` 路径；

6. 两条路径共用同一套索引前缀、标签 sidcar、查询统计接口和测试入口。

对应的路由规则定义为

$$m(F) = |S_F|,$$

$$m(F) \leq \tau_m \Rightarrow \text{prefilter}, \quad m(F) > \tau_m \Rightarrow \text{graph-only}.$$

4 正式代码迁移方案

4.1 迁移原则

迁移时遵循以下原则：

1. hybrid 相关 C++ 逻辑迁入正式源码树，不再长期保留在实验目录中独立编译；
2. hybrid_rebuild_context 只作为迁移阶段的临时对照源使用，不作为长期保留目录；
3. 统一查询入口放入正式查询接口，而不是继续由 Python 脚本决定选择哪一个独立二进制；
4. 当正式 binary、正式测试和阈值标定流程稳定后，整体删除 hybrid_rebuild_context 目录及其构建、脚本、缓存、绘图和辅助文件，不保留实验目录。

4.2 模块落点

建议按下表拆分并迁移模块：

当前来源	正式落点	实施动作
hybrid_rebuild_context/src/search_disk_index_filtered_prefilter.cpp 中的 DenseBitset sidcar 读取、计数与候选物化逻辑	include/filter/densebit_index.h 与 src/filter/densebit_index.cpp	抽出独立 DenseBitset 运行时模块，提供 count_candidates、materialize_candidates、load_sidcar 等接口。
hybrid_rebuild_context/src/search_disk_index_filtered_prefilter.cpp 中的 prefilter PQ shortlist 与 SSD 精排逻辑	src/search/hybrid_prefilter.cpp	抽出正式 prefilter 执行器，使其不依赖实验目录下的 main 函数。
hybrid_rebuild_context 中的临时路由判断	include/filter/hybrid_route.h 与 src/search/hybrid_search.cpp	在正式源码树中实现基于 τ_m 的统一路由器，并由正式查询入口调用。
include/ssd_index.h 与 src/search/pipe_search.cpp 已有的 graph-only 过滤搜索入口	保持在正式源码树	不重写 graph-only 核心图搜索，只新增统一的 hybrid 调度层。

hybrid_rebuild_context	src/CMakeLists.txt 与 中的独立 binary 接线与迁 移辅助入口	tests/CMakeLists.txt	将 hybrid 相关编译接线并 入主工程，补充正式测试驱 动，并在迁移完成后移除对 实验目录的依赖。
------------------------	---	----------------------	--

1. 在 `src/CMakeLists.txt` 中将源码 glob 从当前的 `*.cppsearch/*.cppupdate/*.cpputils/*.cpp` 扩展为包含 `filter/*.cpp`;
2. 在 `include/ssd_index.h` 中声明统一 hybrid 查询接口，例如 `hybrid_search(...)`;
3. 在 `tests/` 中新增正式驱动，例如 `tests/search_disk_index_hybrid.cpp`，用于 smoke 与回归;
4. 当正式 binary、正式测试和阈值标定流程均已稳定后，删除整个 `hybrid_rebuild_context` 目录，并移除所有对该目录的构建、脚本和文档引用。

4.3 实验目录删除时机

`hybrid_rebuild_context` 的删除时机放在迁移收尾阶段，而不是放在模块开发中途。删除前必须同时满足以下条件：

1. `DenseBitset`、`prefilter`、统一 hybrid 查询入口、阈值标定与后台重定位都已迁入正式源码树;
2. 主工程可以独立完成构建、smoke、回归与阈值初始化，不再调用 `hybrid_rebuild_context` 中的 C++ binary、脚本或缓存;
3. 正式测试驱动已经覆盖迁移后的核心路径;
4. 文档中各模块记录已经补齐正式落地实现，能够独立说明正式实现方式。

一旦上述条件满足，就直接整体删除 `hybrid_rebuild_context` 目录，不保留实验副本。

4.4 文档更新规则

本文档需要作为后续正式实现的持续更新接口使用。自本计划开始执行后，每完成一个模块，都必须同步更新本文档中的对应模块记录，至少补齐以下内容：

1. 模块状态，从“未开始”更新为“进行中”或“已完成”;
2. 实际修改的头文件、源码文件、测试文件和构建文件路径;
3. 模块的具体实现方式，包括关键数据结构、控制流与接口设计;
4. 至少一段核心代码片段，用于说明该模块的实际落地方式;
5. 模块验证结果，包括编译、测试、smoke 或回归命令及其结论。

为避免文档退化为简单进度表，后续更新时必须优先补“具体实现方式”和“核心代码片段”，而不仅仅写“已完成”或“已支持”。

5 统一 hybrid 查询入口

统一 hybrid 查询入口建议作为正式查询接口的一部分实现，而不是继续保留为脚本层的路径分发。该入口需要完成以下工作：

1. 接收查询向量、过滤表达式、`k`、`L`、`beamwidth` 等正式参数；
2. 基于当前过滤表达式计算候选规模 $m(F)$ ；
3. 将 $m(F)$ 与当前阈值 τ_m 比较，并记录本次选路结果；
4. 当选择 prefilter 路径时，若已有 bitset 中间结果，则直接复用；
5. 当选择 graph-only 路径时，直接复用现有 `pipe_search(..., selector, filter_data, ...)`；
6. 向统一统计接口输出路径选择、候选规模、阈值版本和查询耗时。

6 候选规模计算机制

6.1 单标签查询

对于单标签查询，正式实现中建议维护每个标签的 live 候选数。若迁移初期仍沿用已有的选择性表，则可按

$$m(F) = \rho(F) \cdot N_{\text{live}}$$

换算出候选规模；当正式元数据补齐后，运行时直接读取该标签的 live 候选数即可。无论使用哪种表示，路由比较对象都统一为 $m(F)$ ，而不是直接比较选择性。

6.2 多标签交集查询

对于多标签交集查询，即要求结果向量至少包含查询中的全部标签，运行时在 DenseBitset sidecar 上执行按位与，再对结果做 `popcount`，得到不重复候选数：

$$m(F) = \left| \bigcap_{\ell \in F} S_\ell \right|.$$

若最终选中 prefilter 路径，则复用同一份 bitset 结果继续物化候选 ID；若最终选中 graph-only 路径，则只保留计数结果参与路由，不额外物化 ID 列表。

6.3 多标签并集查询

对于多标签并集查询，运行时在 DenseBitset sidecar 上执行按位或，再对结果做 `popcount`，得到去重后的候选规模：

$$m(F) = \left| \bigcup_{\ell \in F} S_\ell \right|.$$

这里同样遵循“先计数后选路”的原则。只有在最终选中 prefilter 路径时，才将 bitset 结果物化为候选 ID 列表。

7 阈值初始化与持久化

7.1 初始阈值生成

第一次索引构建完成后，系统立即执行一次轻量 benchmark，用来生成初始候选规模阈值 τ_m 。该 benchmark 仅服务于阈值标定，不承担实验作图职责，要求如下：

1. 使用单线程执行，避免与前台检索竞争资源；
2. 在一组代表性过滤查询上分别运行 prefilter 与 graph-only；
3. 记录两条路径在相同检索参数下的延迟；
4. 找到两条成本曲线的交叉区间，并输出一个初始候选规模阈值 τ_m ；
5. 将 τ_m 、标定时的 live 向量总数 N_{calib} 以及标定时间持久化为正式元数据。

7.2 阈值元数据

建议在正式元数据中增加以下字段：

1. 当前候选规模阈值 τ_m ；
2. 上一次标定时的 live 向量总数 N_{calib} ；
3. 当前 live 向量总数 N_{live} ；
4. 阈值版本号与上次标定时间；
5. 阈值是否处于待重定位状态的标志位。

8 阈值重定位机制

8.1 唯一触发条件

阈值重定位的唯一触发条件定义为 live 向量总数相对上一次标定时的向量总数发生超过 10% 的变化。触发条件写为

$$\frac{|N_{\text{live}} - N_{\text{calib}}|}{N_{\text{calib}}} > 0.10.$$

除这一条件外，不再引入任何其他自动触发条件。标签变化不触发阈值重定位，维护事件也不单独触发阈值重定位。

8.2 后台重定位执行方式

当触发条件成立后，系统只做两件事：

1. 将阈值状态标记为待重定位；
2. 在后台启动单线程轻量 benchmark，重新生成新的 τ_m 并更新 N_{calib} 。

后台重定位期间，前台查询继续使用旧阈值工作，待新阈值生成并持久化后再切换。

8.3 前台负载检查

每条后台 benchmark 查询开始前,都必须检查一次前台负载。若前台处于高负载,则本条 benchmark 查询不执行,后台任务暂停并等待下一个空闲窗口。前台负载检查建议至少包含以下条件:

1. 当前前台查询并发低于低水位;
2. 查询等待队列为空或接近空;
3. 当前没有正在执行的前台高优先级维护任务;
4. 当前没有用户显式禁止后台重定位。

9 更新路径配套要求

由于系统支持更新,更新路径需要同时维护路由输入与阈值元数据:

1. 插入与删除操作必须更新 live 向量总数 N_{live} ;
2. 插入、删除和标签变更必须更新 DenseBitset sidecar 对应的 live 状态,保证 $m(F)$ 的查询期计算正确;
3. 标签变化只更新候选规模计算输入,不触发阈值重定位;
4. 每次更新完成后都要检查一次是否满足 10% 的向量数量变化阈值;
5. 若满足条件,则仅设置后台重定位任务,不阻塞前台查询路径。

10 验证计划

实现完成后,至少执行以下验证:

10.1 构建与集成验证

1. 主工程 `src/CMakeLists.txt` 能成功编译包含 `filter/*.cpp` 的正式库;
2. 正式测试驱动可以直接调用统一 hybrid 查询入口;
3. 主工程中的正式 binary 可以独立完成 smoke 与回归,不再依赖 `hybrid_rebuild_context` 中的 binary 或脚本;
4. 删除 `hybrid_rebuild_context` 后,仓库仍能完成构建与核心验证。

10.2 路由正确性验证

1. 单标签查询可基于 live 候选规模正确选路;
2. 多标签交集查询可通过 bitset 按位与加 `popcount` 正确计算 $m(F)$;
3. 多标签并集查询可通过 bitset 按位或加 `popcount` 正确计算 $m(F)$;
4. 路由决策日志中能看到 $m(F)$ 、 τ_m 和最终路径选择。

10.3 阈值重定位验证

1. 当 N_{live} 与 N_{calib} 的差值超过 10% 时, 后台重定位任务被正确触发;
2. 后台重定位始终以单线程执行;
3. 每条后台 benchmark 查询前都执行前台负载检查;
4. 当负载检查失败时, 后台任务能暂停并在后续继续;
5. 阈值更新后, τ_m 与 N_{calib} 被正确持久化。

11 模块实施记录接口

本节按模块组织, 作为后续持续更新的统一接口。当前版本只给出记录模板; 每个模块正式实现后, 都应在对应小节中补入实际实现说明与核心代码片段。

11.1 模块 M1: DenseBitset 运行时模块

状态: 未开始。

目标文件: 建议落点为 `include/filter/densebit_index.h`、`src/filter/densebit_index.cpp`。

待补实现说明: 完成后在此补充 sidecar 加载、候选计数、候选物化、交并集位运算与查询期复用方式。

待补核心代码片段:

```
// TODO(M1): 在完成 DenseBitset 正式迁移后, 粘贴核心实现代码片段。
// 建议片段内容: sidecar 加载、count_candidates、materialize_candidates。
```

待补验证结果: 完成后在此补充编译结果、单测或 smoke 结果。

11.2 模块 M2: Prefilter 执行器

状态: 未开始。

目标文件: 建议落点为 `src/search/hybrid_prefilter.cpp`, 必要时补充对应头文件。

待补实现说明: 完成后在此补充 PQ shortlist、SSD 原始向量精排、与 bitset 中间结果的对接方式。

待补核心代码片段:

```
// TODO(M2): 在完成 prefilter 执行器正式迁移后, 粘贴核心实现代码片段。
// 建议片段内容: PQ 打分、top-Lr 维护、精确重排主循环。
```

待补验证结果: 完成后在此补充 recall、延迟或 smoke 结果。

11.3 模块 M3: 统一 Hybrid 查询入口与路由器

状态: 未开始。

目标文件: 建议落点为 `include/ssd_index.h`、`include/filter/hybrid_route.h`、`src/search/hybrid_search.cpp`。

待补实现说明：完成后在此补充正式查询入口签名、路由决策流程、与现有 `pipe_search(...)` 的衔接方式。

待补核心代码片段：

```
// TODO(M3): 在完成统一 hybrid 查询入口后, 粘贴核心实现代码片段。
// 建议片段内容: hybrid_search 主入口与 route(query_filter, tau_m) 逻辑。
```

待补验证结果：完成后在此补充单标签、多标签交集、多标签并集选路结果。

11.4 模块 M4: 候选规模计算模块

状态：未开始。

目标文件：建议复用 `include/filter/densebit_index.h` 与 `src/filter/densebit_index.cpp`, 必要时拆出独立 helper。

待补实现说明：完成后在此补充单标签、交集查询、并集查询的候选规模计算路径, 以及“先计数后选路”的复用关系。

待补核心代码片段：

```
// TODO(M4): 在完成候选规模计算模块后, 粘贴核心实现代码片段。
// 建议片段内容: bitset AND/OR + popcount 的计数逻辑。
```

待补验证结果：完成后在此补充候选规模计算与基准结果的一致性检查。

11.5 模块 M5: 初始阈值标定与元数据持久化

状态：未开始。

目标文件：建议落点为正式构建后工具、索引元数据读写模块及相关测试文件。

待补实现说明：完成后在此补充 build 后轻量 benchmark、 τ_m 生成逻辑、 N_{calib} 与阈值版本持久化方案。

待补核心代码片段：

```
// TODO(M5): 在完成阈值初始化模块后, 粘贴核心实现代码片段。
// 建议片段内容: benchmark 主循环、交叉点求解、元数据写回。
```

待补验证结果：完成后在此补充初始阈值生成与元数据落盘结果。

11.6 模块 M6: 后台阈值重定位与前台负载检查

状态：未开始。

目标文件：建议落点为运行时调度模块、后台任务模块及相关测试文件。

待补实现说明：完成后在此补充 10% 向量数量变化触发逻辑、后台单线程 benchmark 调度、每条 benchmark 查询前的前台负载检查流程。

待补核心代码片段：

```
// TODO(M6): 在完成后台重定位模块后, 粘贴核心实现代码片段。
// 建议片段内容: trigger_recalibration_if_needed 与 load guard 逻辑。
```

待补验证结果：完成后在此补充后台暂停、恢复和阈值切换结果。

11.7 模块 M7：更新路径集成

状态：未开始。

目标文件：建议落点为 `src/update/` 相关文件、标签 `sidecar` 维护逻辑及测试文件。

待补实现说明：完成后在此补充 N_{live} 维护、标签变化引起的 `live` 状态更新，以及不阻塞前台的触发接线。

待补核心代码片段：

// TODO(M7)：在完成更新路径集成后，粘贴核心实现代码片段。
// 建议片段内容：`insert/delete` 后的 `live count` 更新与重定位触发检查。

待补验证结果：完成后在此补充更新前后候选规模与阈值状态检查结果。

11.8 模块 M8：正式测试驱动与实验目录删除

状态：未开始。

目标文件：建议落点为 `tests/search_disk_index_hybrid.cpp`、`tests/CMakeLists.txt`、主工程构建配置及仓库中的相关文档引用。

待补实现说明：完成后在此补充正式 `binary` 接线、测试入口设计、删除 `hybrid_rebuild_context` 前后的仓库清理步骤，以及移除实验目录引用的方式。

待补核心代码片段：

// TODO(M8)：在完成正式测试驱动与 `hybrid_rebuild_context` 删除后，粘贴核心实现代码片段。
// 建议片段内容：测试入口 `main`、构建接线或删除实验目录依赖的关键变更。

待补验证结果：完成后在此补充正式测试通过情况，以及删除 `hybrid_rebuild_context` 后仓库仍可正常构建的结果。

12 实施顺序

建议按以下顺序落地：

1. 抽出 `DenseBitset` 运行时模块，迁入正式 `include/filter` 与 `src/filter`；
2. 抽出 `prefilter` 执行器，迁入正式 `src/search`；
3. 在 `include/ssd_index.h` 和正式查询入口中加入统一 `hybrid` 查询接口；
4. 在主工程 `CMake` 和 `tests` 中接入正式 `hybrid binary`；
5. 实现构建后轻量 `benchmark`，生成初始 τ_m ；
6. 接入更新路径中的 N_{live} 维护与 10% 触发逻辑；
7. 实现后台单线程重定位与前台负载检查；
8. 最后在正式 `binary`、正式测试和阈值流程稳定后，整体删除 `hybrid_rebuild_context` 目录及其所有残余引用。

13 交付结果

本计划完成后，工程侧应交付以下内容：

1. 正式源码树中的 hybrid 查询实现；
2. 正式构建系统中的 hybrid 编译接线；
3. 正式测试驱动与回归验证；
4. 构建后初始阈值标定流程；
5. 仅由向量数量变化 10% 触发的后台阈值重定位流程；
6. 仓库中不再保留 `hybrid_rebuild_context` 目录。